1η Εργαστηριακή Άσκηση

Ψηφιακά Συστήματα VLSI

Παναγιώτης Γερασιμόπουλος 03115208
personal@devcol.com

Ακαδημαϊκό Έτος 2025-2026

Contents

1. Μέρος Α 1
   1.1. Decoder 3-8 1
      1.1.1. Implementation 1
         1.1.1.1. Dataflow 1
         1.1.1.2. Behavioral 1
      1.1.2. Testbench 2
   1.2. Καταχωρητής Ολίσθησης 3
      1.2.1. Implementation 3
      1.2.2. Test Bench 4
   1.3. Μετρητής 6
      1.3.1. Implementation 6
      1.3.2. Test Bench 8
2. Μέρος Β 10
   2.1. Half Adder 10
      2.1.1. Implementation 10
      2.1.2. Test Bench 11
   2.2. Full Adder 11
      2.2.1. Implementation 12
      2.2.2. Test Bench 13
   2.3. 4 Bit Full Adder 14
      2.3.1. Implementation 14
      2.3.2. Test Bench 14
   2.4. BCD 15
      2.4.1. Implementation 15
      2.4.2. Test Bench 16
   2.5. 4 Digit BCD 17
      2.5.1. Implementation 17
      2.5.2. Test Bench 18

1. Μέρος Α

1.1. Decoder 3-8

Ο αποκωδικοποιητής 3 σε 8 είναι ένα συνδυαστικό κύκλωμα που για κάθε συνδυασμό των εισόδων του ενεργοποιεί μόνο μία από τις εξόδους.

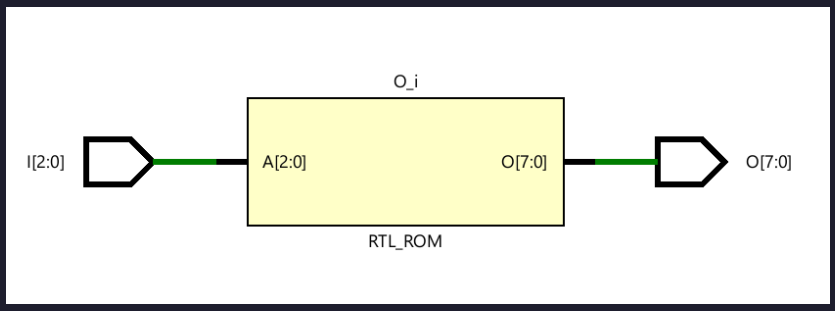

Figure 2: RTL

1.1.1. Implementation

Ο πίνακας αληθείας του αποκωδικοποιητή φαίνεται από τις γραμμές 16-23 του κωδικά για το behavioral.

1.1.1.1. Dataflow

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Decoder3x8 is
 port (
   I : in  std_logic_vector(2 downto 0);
   O : out std_logic_vector(7 downto 0));
end entity;

architecture Dataflow of Decoder3x8 is
begin
  with I select O <=
  "00000001"      when "000",
  "00000010"      when "001",
  "00000100"      when "010",
  "00001000"      when "011",
  "00010000"      when "100",
  "00100000"      when "101",
  "01000000"      when "110",
  "10000000"      when "111",
  (others => '-') when others;
end Dataflow;
```

1.1.1.2. Behavioral

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity Decoder3x8 is
 port (
   I : in  std_logic_vector(2 downto 0);
   O : out std_logic_vector(7 downto 0));
end entity;

architecture Behavioral of Decoder3x8 is
begin
  process(I)
  begin
    case I is
      when "000"  => O <= "00000001";
      when "001"  => O <= "00000010";
      when "010"  => O <= "00000100";
      when "011"  => O <= "00001000";
      when "100"  => O <= "00010000";
      when "101"  => O <= "00100000";
      when "110"  => O <= "01000000";
      when "111"  => O <= "10000000";
      when others => O <= (others => '-');
    end case;
  end process;
end Behavioral;
```

1.1.2. Testbench

Το κύκλωμα έχει πολύ λίγες πιθανές εισόδους και δεν έχει state οπότε μπορούμε εύκολα να σχεδιάσουμε εξαντλητικά test bench όπως φαίνεται παρακάτω.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Bench is
end entity;

architecture Behavioral of Bench is
  component Decoder3x8 is
  port (
    I   : in  std_logic_vector(2 downto 0);
    O   : out std_logic_vector(7 downto 0)
  );
  end component;
signal I: std_logic_vector(2 downto 0);
signal O: std_logic_vector(7 downto 0);

begin
  uut: Decoder3x8 port map (I => I, O => O);

  process begin

    for j in 0 to 7 loop
      I <= std_logic_vector(to_unsigned(j, 3));
      wait for 10 ns;
```

```
      end loop;
    wait;

  end process;
end architecture;
```

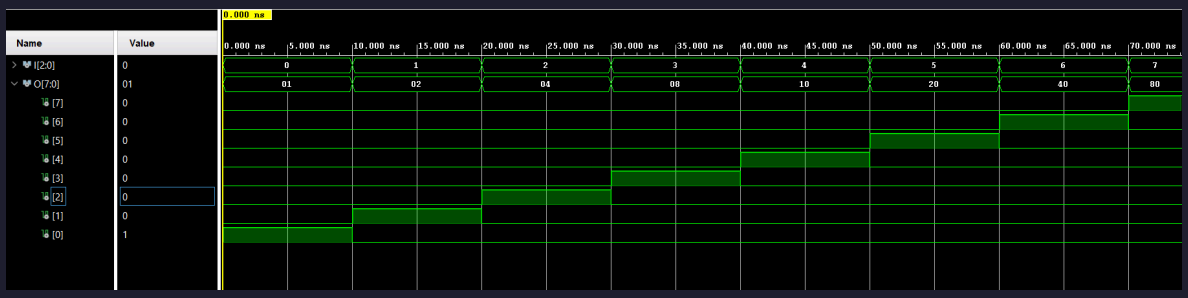

Figure 3: Test Bench

1.2. Καταχωρητής Ολίσθησης

Ο καταχτητής ολίσθησης είναι σύγχρονο κύκλωμα που δέχεται μια παράλληλη είσοδο των 4bit μέσω του I και μια συριακή μέσω του SerIn. Τα σήματα έλεγχου του είναι το reset που τον επαναφέρει στο 0 ασύγχρονα, το load που ελέγχει την παράλληλη είσοδο, το enable που ελέγχει την σειριακή λειτουργία του κυκλώματος και το slide bit που ελέγχει την κατεύθυνση της ολίσθησης ασύγχρονα.

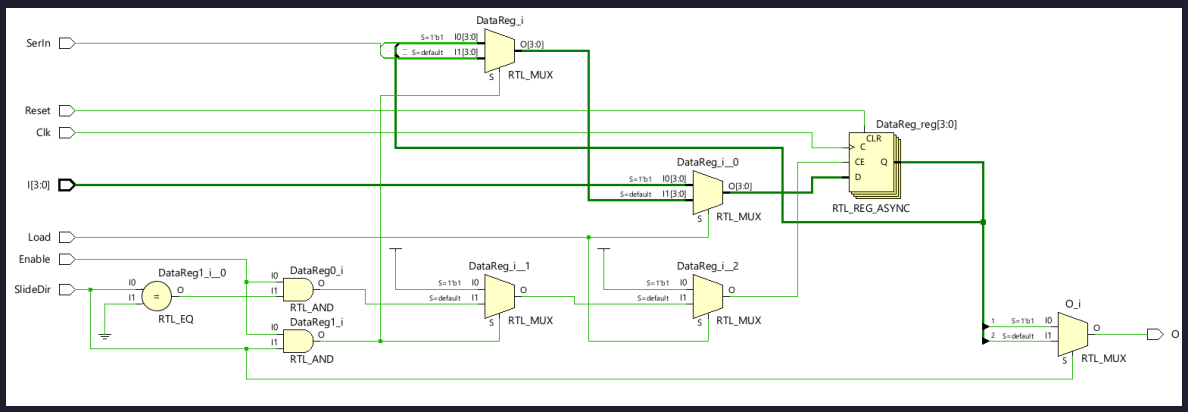

Figure 4: RTL

1.2.1. Implementation

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ShiftRegister is
  Port(
    Clk, Reset, Load, Enable, SerIn, SlideDir: in std_logic;
    I : in std_logic_vector(3 downto 0);
    O : out std_logic
  );
end entity;
```

```
architecture zzz of ShiftRegister is
  signal DataReg: std_logic_vector(3 downto 0);
begin

  edge:process(Clk, Reset) begin
    if Reset = '1' then
      DataReg <= (others => '0');
    elsif rising_edge(Clk) then
      if Load = '1' then
        DataReg <= I;
      elsif Enable = '1' and SlideDir = '1' then
        DataReg <= SerIn & DataReg(3 downto 1);
      elsif Enable = '1' and SlideDir = '0' then
        DataReg <= DataReg(2 downto 0) & SerIn;
      end if;
    end if;
  end process;

  O <= DataReg(0) when SlideDir = '1' else DataReg(3);

end zzz;
```

Το οποίο μας δίνει στο critical path του synthesis 5.351ns.

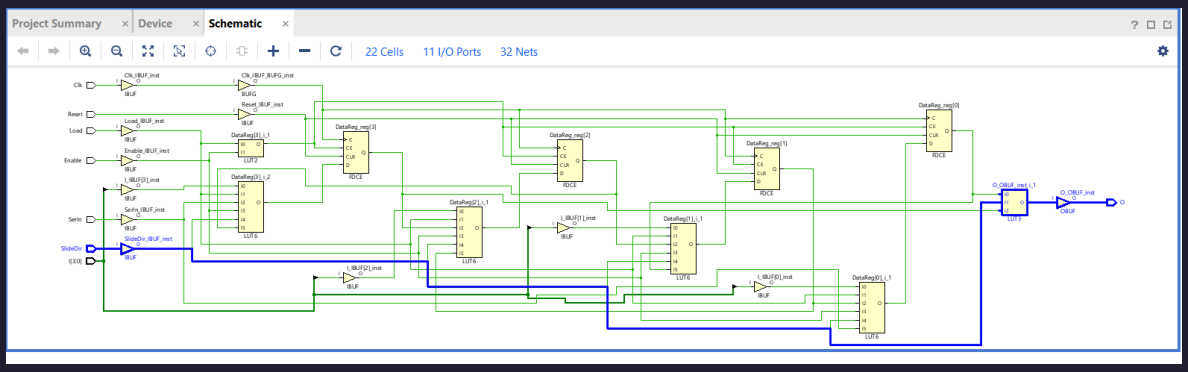

Figure 5: Critical Path 5.351ns

1.2.2. Test Bench

Φορτώνουμε τον αριθμό '1010' μέσω του load και κάνουμε αριστερό slide για 3 cycles φορτώνοντας 4 '1' μέσω του SerIn και μετά κάνουμε δυο cycles slide δεξιά περιμένοντας να δούμε τους άσους που φορτώσαμε στα register. Τα input δίνονται στο falling edge ενώ το σύστημα κάνει update στο rising edge.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Tb is
end entity;

architecture zzz of Tb is
  component ShiftRegister
   Port(
     Clk, Reset, Load, Enable, SerIn, SlideDir: in std_logic;
     I: in std_logic_vector(3 downto 0);
```

```
     O: out std_logic
    );
  end component;

  signal Clk:       std_logic := '0';
  signal Load:      std_logic;
  signal Reset:     std_logic;
  signal Enable:    std_logic;
  signal SlideDir:  std_logic;
  signal SerIn:     std_logic;
  signal I:         std_logic_vector(3 downto 0);
  signal O:         std_logic;
  constant CLOCK_PERIOD : time := 10 ns;

begin
  Dut: ShiftRegister port map (
    Clk       => Clk,
    Load      => Load,
    Reset     => Reset,
    Enable    => Enable,
    SlideDir  => SlideDir,
    SerIn     => SerIn,
    I         => I,
    O         => O
  );

  Clk <= not Clk after CLOCK_PERIOD / 2;

  Test: process begin
    Load <= '1';
    Reset <= '0';
    Enable <= '1';
    SlideDir <= '0';
    SerIn <= '1';
    I <= "1010";

    wait until falling_edge(Clk);
    Load <= '0';
    wait until falling_edge(Clk);
    wait until falling_edge(Clk);
    wait until falling_edge(Clk);
    SerIn <= '0';
    SlideDir <='1';
    wait until falling_edge(Clk);
    wait until falling_edge(Clk);
    Reset<='1';

    wait;
  end process;
end architecture;
```

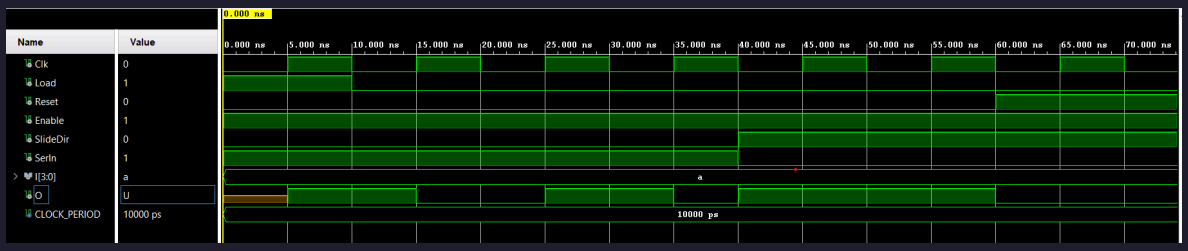

Figure 6: Test Bench

1.3. Μετρητής

Ο Μετρητής είναι ένα σύγχρονο κύκλωμα που μετράει σε διάδικο τους κύκλους του ρολογιού και κάνει wrap στο overflow, στην περίπτωση μας μετά τον αριθμό 7. Το ResetN μηδενίζει την έξοδο ασύγχρονα, το down ελέγχει αν ο μετρητής θα αυξάνει η μειώνει σε κύκλο, και το CountEn λειτουργεί ως start stop της μέτρησης. Επίσης προσδέθηκαν τα Modulo και Up που ελέγχουν το σημείο που κάνουμε wrap και την κατεύθυνση της μέτρησης.

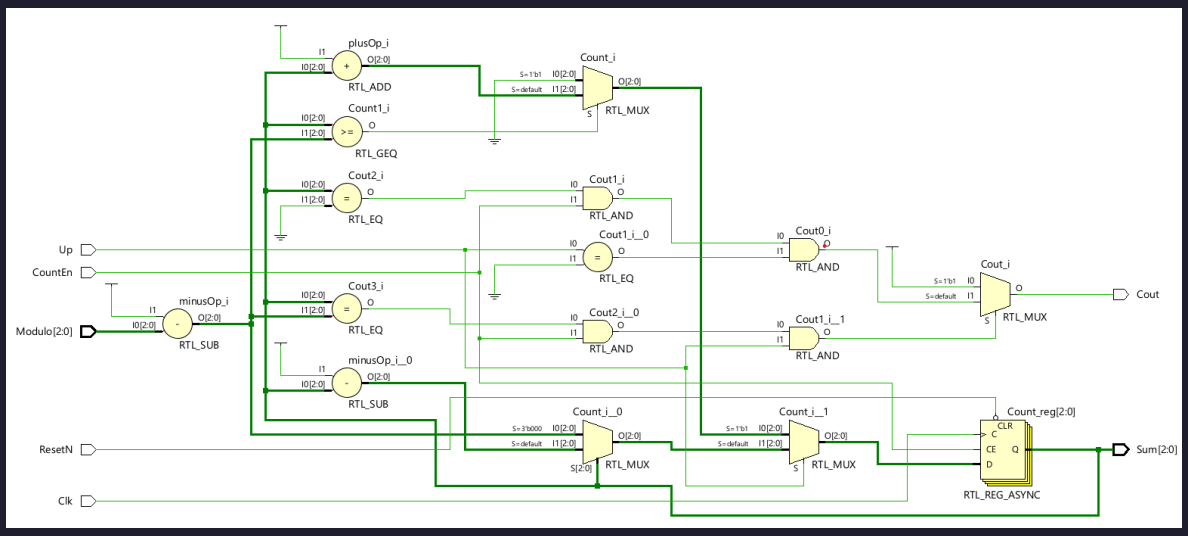

Figure 7: RTL

1.3.1. Implementation

Και τα δυο μέρη της άσκησης έγιναν σε ένα κύκλωμα για ευκολία στο debug, το κύκλωμα υποστηρίζει dynamic αλλαγή της τιμής του modulo αλλά μόνο προς το θετικό count (αφού δεν ζητήθηκε modulo για μέτρηση προς τα κάτω).

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Counter is
  Port (
    Clk,
    ResetN,
    Up,
    CountEn : in std_logic;
    Modulo : in std_logic_vector(2 downto 0);
```

```
    Sum : out std_logic_vector(2 downto 0);
    Cout : out std_logic
  );
end Counter;

architecture zzz of Counter is
signal Count : unsigned(2 downto 0);
begin

  process(Clk, ResetN) begin
    if ResetN = '0' then
      Count <= (others =>'0');
    elsif  (rising_edge(Clk)) then
      if CountEn = '1' then

        if Up = '1' then
          -- Up Count
          if Count >= unsigned(Modulo) - 1 then
            Count <= (others => '0');
          else
            Count <= Count + 1;
          end if;

        else
          -- Down Count
          if Count = 0 then
            Count <= unsigned(Modulo) - 1;
          else
            Count <= Count - 1;
          end if;
        end if;

      end if;
    end if;
  end process;

  Sum <= std_logic_vector(Count);
  Cout <= '1' when (Count = unsigned(Modulo)-1 and CountEn='1' and Up='1')
else
          '1' when (Count = 0 and CountEn = '1' and Up = '0') else
          '0';

end architecture;
```

Το οποίο μας δίνει στο critical path του synthesis 5.924ns.

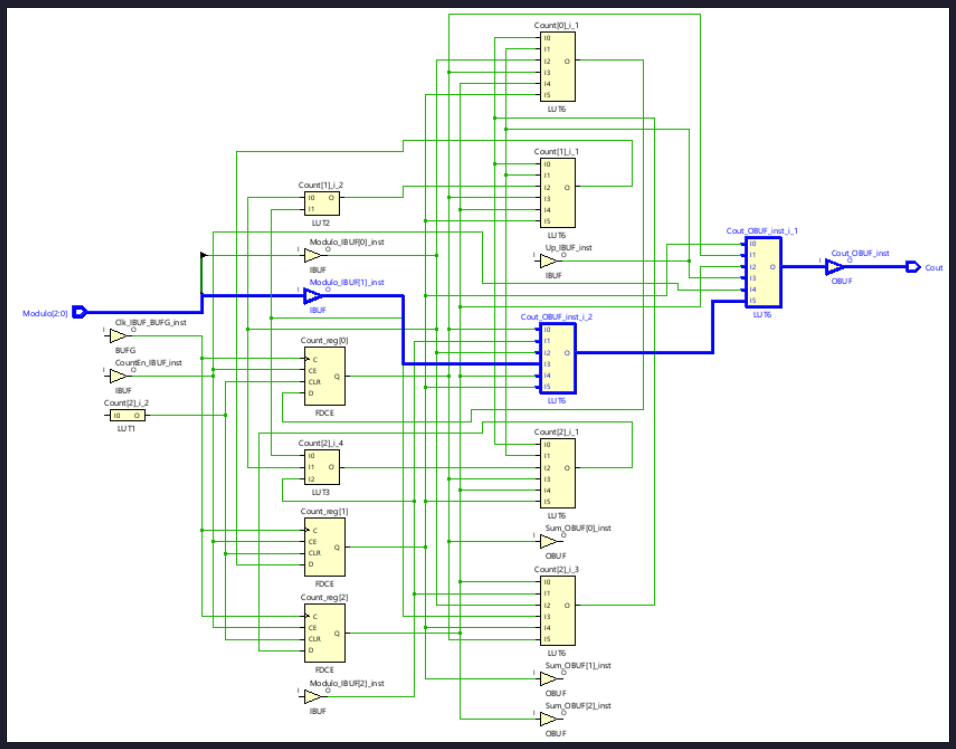

Figure 8: Critical Path 5.924ns

1.3.2. Test Bench

Για το test bench κάνουμε αρχικά μέτρηση προς τα πάνω μέχρι overflow και μετά προς τα κάτω μέχρι underflow. Στην συνέχεια αλλάζουμε το modulo και δοκιμάζουμε την δυναμική αλλαγή του στην μέτρηση προς τα πάνω.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Counter_Tb is
end Counter_Tb;

architecture Simulation of Counter_Tb is
  component Counter
    Port (
      Clk     : in std_logic;
      ResetN  : in std_logic;
      Up      : in std_logic;
      CountEn : in std_logic;
      Modulo  : in std_logic_vector(2 downto 0);
      Sum     : out std_logic_vector(2 downto 0);
      Cout    : out std_logic
    );
  end component;

  signal Clk     : std_logic := '0';
  signal ResetN  : std_logic := '0';
  signal Up      : std_logic := '1';
  signal CountEn : std_logic := '0';
  signal Modulo  : std_logic_vector(2 downto 0) := "101";
```

```
  signal Sum     : std_logic_vector(2 downto 0);
  signal Cout    : std_logic;
  constant CLK_PERIOD : time := 10 ns;

begin
  DUT: Counter port map (
    Clk     => Clk,
    ResetN  => ResetN,
    Up      => Up,
    CountEn => CountEn,
    Modulo  => Modulo,
    Sum     => Sum,
    Cout    => Cout
  );

  Clk <= not Clk after CLK_PERIOD / 2;

  process begin
    ResetN <= '0';
    wait for CLK_PERIOD * 2;
    ResetN <= '1';
    -- NOTE(acol): Up count with wrap
    CountEn <= '1';
    Up <= '1';
    Modulo <= "101";
    wait for CLK_PERIOD * 7;
    -- NOTE(acol): Down count with wrap
    Up <= '0';
    wait for CLK_PERIOD * 4;
    -- NOTE(acol): Modulo dynamic change
    Modulo <= "011";
    Up <= '1';
    wait for CLK_PERIOD * 5;
    -- NOTE(acol): Disable
    CountEn <= '0';
    wait for CLK_PERIOD * 3;

    wait;
  end process;
end Simulation;
```

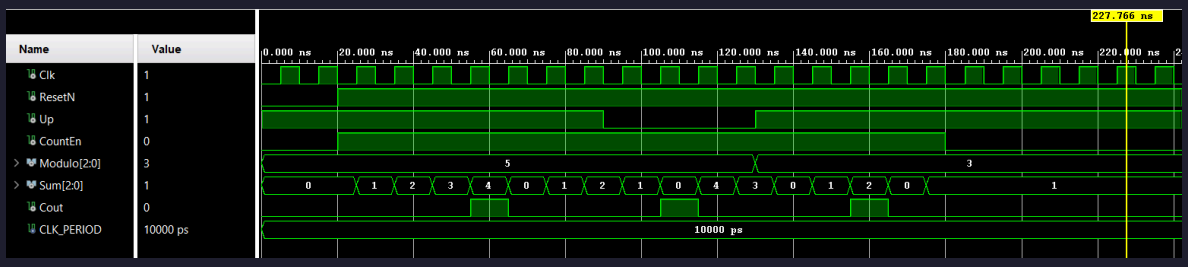

Figure 9: Test Bench

2. Μέρος B

2.1. Half Adder

Ο half adder είναι ένα πολύ απλό κύκλωμα που περιγράφεται εξολοκλήρου από τις λογικές συναρτήσεις $C_{out} = A \wedge B$ και $S = A \oplus B$.

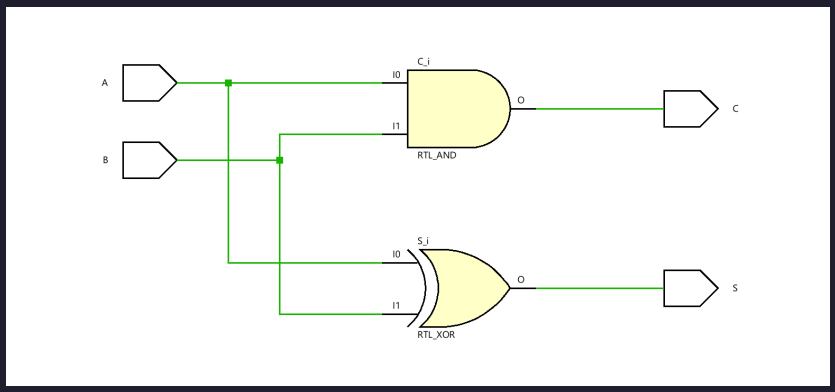

Figure 10: RTL

2.1.1. Implementation

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity HalfAdder is
  Port(
    A, B: in std_logic;
    S, C: out std_logic
  );
end entity;

architecture Dataflow of HalfAdder is
begin
  C <= A and B;
  S <= A xor B;
end architecture;
```

Το οποίο μας δίνει στο critical path του synthesis 5.377ns.

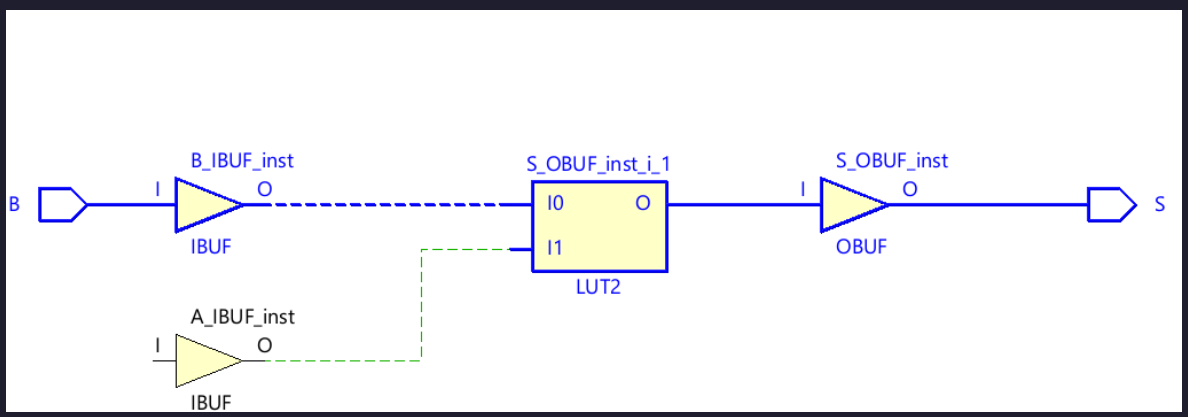

Figure 11: Critical Path 5.377

2.1.2. Test Bench

Μπορούμε να εξετάσουμε διεξοδικά τον πίνακα αληθείας.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity HalfAdder_Tb is
end HalfAdder_Tb;

architecture Simulation of HalfAdder_Tb is
  component HalfAdder
    Port (
      A, B : in std_logic;
      S, C : out std_logic
    );
  end component;

  signal A, B : std_logic := '0';
  signal S, C : std_logic;

begin

  DUT: HalfAdder port map (
    A => A,
    B => B,
    S => S,
    C => C
  );

  process begin
    a <= '0'; b <= '0';
    wait for 10 ns;
    a <= '0'; b <= '1';
    wait for 10 ns;
    a <= '1'; b <= '0';
    wait for 10 ns;
    a <= '1'; b <= '1';
    wait;
  end process;
end architecture;
```

Figure 12: Test Bench

2.2. Full Adder

Για να δημιουργήσουμε έναν Full Adder χρησιμοποιούμε δυο Half Adder σε σειρά και συνδέουμε τα C_{out} με μια πύλη or.

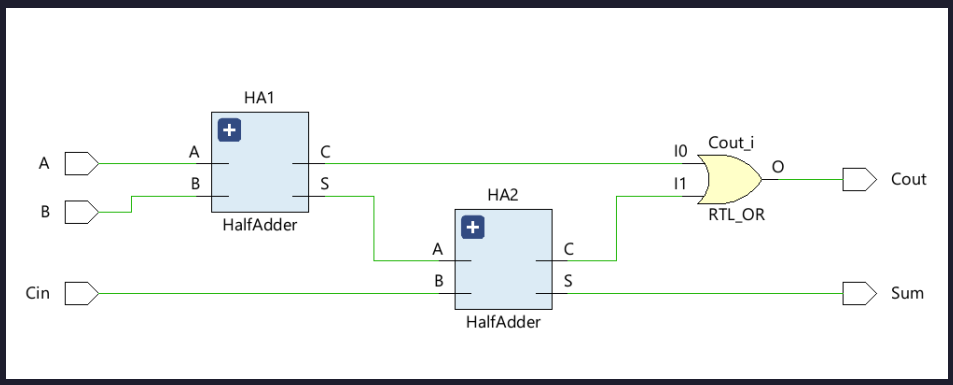

Figure 13: RTL

2.2.1. Implementation

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FullAdder is
  Port (
    A, B, Cin  : in  std_logic;
    Sum, Cout  : out std_logic
  );
end FullAdder;

architecture Structural of FullAdder is
  component HalfAdder
    Port (
      A, B : in  std_logic;
      S, C : out std_logic
    );
  end component;

  signal S1, C1, C2 : std_logic;
begin

  HA1: HalfAdder port map (A => A, B => B, S => S1, C => C1);
  HA2: HalfAdder port map (A => S1, B => Cin, S => Sum, C => C2);
  Cout <= C1 or C2;

end architecture;
```

Το οποίο μας δίνει στο critical path του synthesis 5.377ns, το ίδιο με τον half adder.

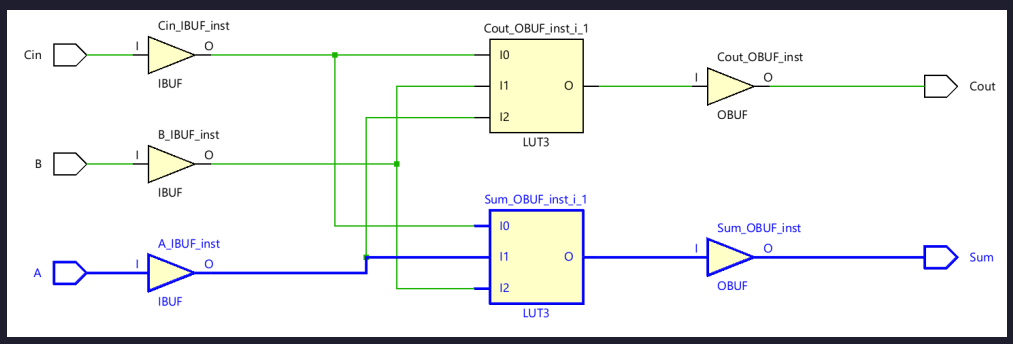

Figure 14: Critical Path 5.377

2.2.2. Test Bench

Μπορούμε να εξετάσουμε διεξοδικά τον πίνακα αληθείας.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity FullAdder_Tb is
end FullAdder_Tb;

architecture Simulation of FullAdder_Tb is
  component FullAdder
    Port (
      A, B, Cin  : in  std_logic;
      Sum, Cout  : out std_logic
    );
  end component;

  signal A, B, Cin : std_logic := '0';
  signal Sum, Cout : std_logic;
begin

  DUT: FullAdder port map (
    A => A,
    B => B,
    Cin => Cin,
    Sum => Sum,
    Cout => Cout
  );

  process
    variable TestVector : unsigned(2 downto 0);
  begin
    for i in 0 to 7 loop
      TestVector := to_unsigned(i, 3);
      A   <= TestVector(2);
      B   <= TestVector(1);
      Cin <= TestVector(0);
      wait for 10 ns;
    end loop;

    wait;

  end process;
end architecture;
```

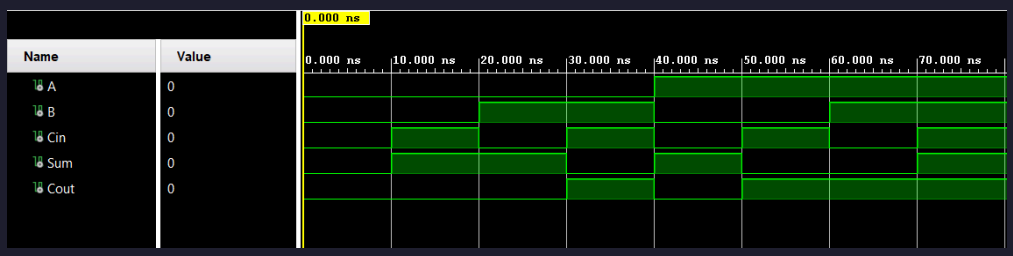

Figure 15: Test Bench

2.3. 4 Bit Full Adder

Υποθέτουμε εδώ ότι ο αθροιστής που ζητείται είναι τύπου ripple carry. Αρκεί να συνδέσουμε σε σειρά τα carry τεσσάρων full adder.

2.3.1. Implementation

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FullAdder4 is
  Port (
    A, B : in  std_logic_vector(3 downto 0);
    Cin  : in  std_logic;
    Sum  : out std_logic_vector(3 downto 0);
    Cout : out std_logic
  );
end FullAdder4;

architecture Structural of FullAdder4 is
  component FullAdder
    Port (
      A, B, Cin : in  std_logic;
      Sum, Cout : out std_logic
    );
  end component;

  signal C : std_logic_vector(4 downto 0);

begin
  C(0) <= Cin;
  Cout <= C(4);

  Gen_FA for i in 0 to 3 generate
    FAInstance: FullAdder port map (
      A    => A(i),
      B    => B(i),
      Cin  => C(i),
      Sum  => Sum(i),
      Cout => C(i+1)
    );
  end generate;
end architecture;
```

2.3.2. Test Bench

Μπορούμε να εξετάσουμε διεξοδικά τον πίνακα αληθείας.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity FullAdder4_Tb is
```

```
end FullAdder4_Tb;

architecture Simulation of FullAdder4_Tb is

  component FullAdder4
    Port (
      A, B : in  std_logic_vector(3 downto 0);
      Cin  : in  std_logic;
      Sum  : out std_logic_vector(3 downto 0);
      Cout : out std_logic
    );
  end component;

  signal A, B : std_logic_vector(3 downto 0) := (others => '0');
  signal Cin  : std_logic := '1';
  signal Sum  : std_logic_vector(3 downto 0);
  signal Cout : std_logic;
begin

  DUT: FullAdder4 port map (
    A    => A,
    B    => B,
    Cin  => Cin,
    Sum  => Sum,
    Cout => Cout
  );

  process begin
    for i in 0 to 15 loop
      for j in 0 to 15 loop

          A <= std_logic_vector(to_unsigned(i, 4));
          B <= std_logic_vector(to_unsigned(j, 4));
          wait for 10 ns;

        end loop;
      end loop;
    end loop;
    wait;
  end process;
end Simulation;
```

2.4. BCD

Το κύκλωμα κάνει πρόσθεση δυο δεκαδικών αριθμών σε δυαδική αναπαράσταση με χρήση δυο Full Adder των 4bit. Ανιχνεύει όταν το άθροισμα είναι πάνω από 9 και προσθέτει 6 στον δεύτερο Full Adder 4bit αλλιώς προσθέτει 0.

2.4.1. Implementation

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity BCD is
  Port (
    A, B : in  std_logic_vector(3 downto 0);
    Cin  : in  std_logic;
    Sum  : out std_logic_vector(3 downto 0);
    Cout : out std_logic
  );
end BCD;

architecture Structural of BCD is
  component FullAdder4 is
    Port (
      A, B : in  std_logic_vector(3 downto 0);
      Cin  : in  std_logic;
      Sum  : out std_logic_vector(3 downto 0);
      Cout : out std_logic
    );
  end component;

  signal C, Cint : std_logic;
  signal A2, B2  : std_logic_vector(3 downto 0);

begin
  Adder1 : FullAdder4 port map (A, B, Cin, A2, C);

  Cint <= C or (A2(3) and A2(2)) or (A2(3) and A2(1));
  Cout <= Cint;

  B2(0) <= '0';
  B2(1) <= Cint;
  B2(2) <= Cint;
  B2(3) <= '0';

  Adder2 : FullAdder4 port map (A2, B2, '0', Sum, open);
end architecture;
```

2.4.2. Test Bench

Το Test Bench είναι παρόμοιο με του 4bit full adder μόνο που ελέγχουμε και για διάφορες τιμές του Cin.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity BCD_Tb is
end BCD_Tb;

architecture Simulation of BCD_Tb is
  component BCD
    Port (
      A, B : in  std_logic_vector(3 downto 0);
      Cin  : in  std_logic;
      Sum  : out std_logic_vector(3 downto 0);
      Cout : out std_logic
```

```
    );
  end component;

  signal A, B : std_logic_vector(3 downto 0) := (others => '0');
  signal Cin  : std_logic := '0';
  signal Cout : std_logic;
  signal Sum  : std_logic_vector(3 downto 0);
begin

  DUT: BCD port map (
    A    => A,
    B    => B,
    Cin  => Cin,
    Sum  => Sum,
    Cout => Cout
  );

  process begin
    for i in 0 to 9 loop
      for j in 0 to 9 loop
        for k in 0 to 1 loop

          A   <= std_logic_vector(to_unsigned(i, 4));
          B   <= std_logic_vector(to_unsigned(j, 4));
          Cin <= k;
          wait for 10 ns;

        end loop;
      end loop;
    end loop;
    wait;
  end process;
end Simulation;
```

2.5. 4 Digit BCD

Για τον αθροιστή 4 δεκαδικών αριθμών θα χρησιμοποιήσουμε 4 κυκλώματα BCD και θα ενώσουμε σε σειρά τα C_{out} τους με ανάλογο τρόπο του 4bit full adder.

2.5.1. Implementation

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity BCD4 is
  Port (
    A, B : in  std_logic_vector(15 downto 0);
    Cin  : in  std_logic;
    Sum  : out std_logic_vector(15 downto 0);
    Cout : out std_logic
  );
end BCD4;
```

```
architecture Structural of BCD4 is
  component BCD is
    Port (
      A    : in  std_logic_vector(3 downto 0);
      B    : in  std_logic_vector(3 downto 0);
      Cin  : in  std_logic;
      Sum    : out std_logic_vector(3 downto 0);
      Cout : out std_logic
    );
  end component;

  signal C : std_logic_vector(4 downto 0);
begin
  C(0) <= Cin;
  Cout <= C(4);

  Gen_BCD: for i in 0 to 3 generate
    BCDInstance: BCD port map (
      A    => A(i * 4 + 3 downto i * 4),
      B    => B(i * 4 + 3 downto i * 4),
      Cin  => C(i),
      Sum  => S(i * 4 + 3 downto i * 4),
      Cout => C(i + 1)
    );
  end generate Gen_BCD;
end Structural;
```

2.5.2. Test Bench

Δεν είναι δυνατό να ελέγξουμε όλες τις πιθανές τιμές, άρα θα κάνουμε probe edge cases.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity BCD4_Tb is
end BCD4_Tb;

architecture Simulation of BCD4_Tb is
  component BCD4
    Port (
      A, B : in  std_logic_vector(15 downto 0);
      Cin  : in  std_logic;
      Sum  : out std_logic_vector(15 downto 0);
      Cout : out std_logic
    );
  end component;

  signal A, B : std_logic_vector(15 downto 0) := (others => '0');
  signal Cin  : std_logic := '0';
  signal Sum  : std_logic_vector(15 standard_logic_vector);
  signal Cout : std_logic;
begin

  DUT: BCD4 port map (A, B, Cin, Sum, Cout);
```

```
  process begin
    -- Sum = x"5555", Cout = '0'
    A <= x"1234"; B <= x"4321"; Cin <= '0';
    wait for 10 ns;
    -- Sum = x"0010", Cout = '0'
    A <= x"0009"; B <= x"0001"; Cin <= '0';
    wait for 10 ns;
    -- Sum = x"0000", Cout = '1'
    A <= x"9999"; B <= x"0001"; Cin <= '0';
    wait for 10 ns;
    -- Sum = x"7776", Cout = '1'
    A <= x"8888"; B <= x"8888"; Cin <= '0';
    wait for 10 ns;
    -- Sum = x"2346", Cout = '0'
    A <= x"1111"; B <= x"1234"; Cin <= '1';
    wait for 10 ns;
    wait;
  end process;
end Simulation;
```